서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험II

실험 전 보고서

STM32 UART 통신 및 FPGA 개별 실습

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 10월 26일
대상	STM32 과제5 · STM32 과제6 · STM32 과제7 · 서보모터 제어 · SRAM 읽기 · 쓰기 · DAC
검증환경	Windows 11 / VS Code / STM32 CMake / Icarus Verilog / VaporView

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: https://github.com/a40093632-ai/ece2-lab6

제출 태그: lab6-pre-v1

커밋 해시: 55e0cd5c1fb2b9a8817b584bf5e96660f86852fa

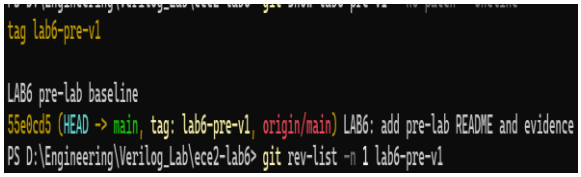


그림 1: GitHub 제출 태그 및 기준 커밋 확인

1. 실험 목적 및 범위

실험 목적

본 실험의 목적은 PC에서 입력한 문자를 STM32가 수신하고, 이를 FPGA로 전달하는 UART 통신 경로를 구성하는 것이다. 먼저 MCU가 수신한 문자와 바이트 값을 확인하고, FPGA의 UART 출력으로 전달 결과를 비교한다. 이후 FPGA의 수신 처리를 확장하여 문자 명령에 따라 LED 상태를 제어한다.

STM32 과제5에서는 PC에서 받은 문자와 해당 문자의 16진수 바이트 값을 출력한다. 과제6에서는 받은 한 바이트를 UART3로 FPGA에 전달하고, FPGA가 이를 PC로 다시 출력하도록 한다. 과제7에서는 수신한 문자를 명령으로 해석하여 FPGA LED를 끄거나 켜고, 현재 상태를 반전하도록 구현한다.

FPGA 개별 실습에서는 서보모터 제어, SRAM 읽기·쓰기, DAC 실습을 순서대로 준비한다. 제공된 모듈별 RTL과 테스트벤치를 이용하여 펄스 생성, 양방향 버스 제어, 디지털 데이터 출력의 관계를 확인한다. 각 회로의 PASS 결과와 대표 파형을 확보하고, 실제 보드에서 확인할 동작을 정리한다.

실험 범위

	실습 내용	사전 검증 방법
과제5	입력 문자와 16진수 바이트 값 출력	CMake 빌드
과제6	MCU의 UART 전달과 FPGA Echo	CMake 빌드 및 RTL 시뮬레이션
과제7	UART 명령에 따른 FPGA LED 제어	CMake 빌드 및 RTL 시뮬레이션
FPGA 실습1	서보 위치 명령과 PWM 펄스 생성	RTL 시뮬레이션
FPGA 실습2	SRAM 저장, 읽기와 버스 방향 제어	RTL 시뮬레이션
FPGA 실습3	8비트 Ramp 데이터의 DAC 인터페이스 출력	RTL 시뮬레이션

2. 관련 이론

DAC의 변환 원리와 분해능

DAC(Digital-to-Analog Converter)는 디지털 입력값을 아날로그 전압이나 전류로 변환하는 회로이다. 이진 가중 방식과 R–2R 저항 사다리 구조는 대표적인 DAC 구현 방식이다. 각 비트는 서로 다른 가중치로 출력에 기여하며, 기준 전압과 입력 코드에 따라 아날로그 출력이 결정된다.[1]

이상적인 단극성 전압 출력 DAC의 관계는 다음과 같다.

$$V_{out} = \frac{D}{2^N} V_{ref}$$

여기서 N 은 비트 수, D는 입력 코드, V는 기준전압이다. 8비트에서는 256개의 출력 수준을 표현하며, 입력 값이 1 증가할 때의 이상적인 전압 변화량인 1 LSB는 V/256이다. 실제 보드의 출력 전압과 극성은 기준전압 및 출력 증폭 회로에 따라 달라진다.

3. 개발 및 검증 환경

개발환경

항목	사용 환경
운영체제	Windows 11 64-bit
편집기	VS Code 1.137.0
파형 확인	VaporView
FPGA Part	Xc7s75fgga484-1
FPGA 도구	AMD Vivado 2026.1
보드	Combo II-DLD S75
STM32 빌드	CMake 기반 STM32
STM32 대상	NUCLEO-G474RE

주요 연결과 파일 역할

대상	역할 및 연결
MCU UART2	ST-LINI VCP를 통한PC 문자 입출력
MCU PC10	UART3 TX, FPGA RX로 문자 전달
MCU PC11	FPGA 응답을 MCU가 받도록 추가 구현할 때 사용하는 RX
FPGA 확장 GPIO RX	MCU PC10에서 보낸 문자 수신
FPGA F6	보드 내부 USB-UART를 통한 PC 출력
GND	MCU와 FPGA 사이 공통 접지

4. STM32 과제 5

설계 목표

PC에서 보낸 문자 한 개를 MCU가 받고, 받은 문자와 해당 바이트 값을 16진수로 VCP UART에 출력한다. Serial Monitor는 9600 8N1로 설정하고, 한 문자 확인 시 전송 줄바꿈은 없음(None)으로 선택한다.

핵심 설계

polling_routine()에서 getchar()의 반환 값을 int형 변수 ch에 저장한다. getchar()는 EOF도 표현할 수 있으므로 결과를 바로 8비트 변수에 저장하지 않는다. ch != EOF이면 이를 uint8_t로 변환하여 한 바이트 값으로 저장한다. 이후 printf()의 %c로 문자를, 0x%02X로 두 자리 16진수 바이트 값을 출력한다. 입력이 없으면 getchar()에서 기다리므로, 입력 대기 상태를 프로그램 오류나 비정상 정지와 구분한다.

예상 결과

PC 입력	예상 문자 출력	예상 바이트 값
1	1	0x31
a	a	0x41
a	a	0x61

사전 검증 결과

문자 입력과 형식화 출력을 포함한 과제5 프로그램이 정상적으로 컴파일·링크되었으며, 빌드 종료 코드 0을 확인하였다. 실제 PC 입력에 대한 Serial Monitor 출력은 보드 실험에서 확인한다.

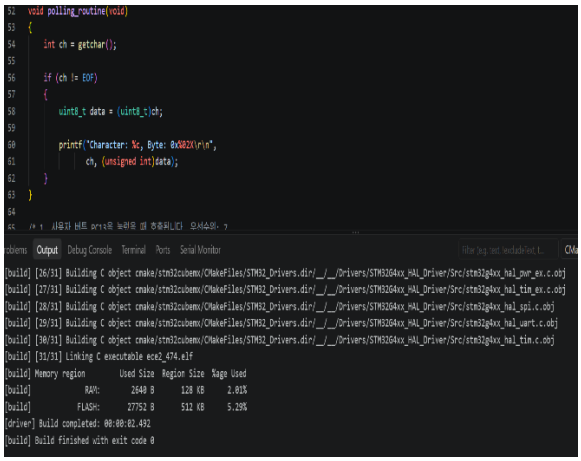


그림 2: STM32 과제 5 코드 및 CMake 빌드 결과

링크: 과제5 소스 코드

링크: 과제5 빌드 및 핵심 코드 증빙

5. STM32 과제 6

설계 목표

다음 경로에서 FPGA가 받은 문자가 원래 PC 입력과 일치하는지 확인한다.

PC – MCU VCP – MCU UART3 – FPGA – USB UART -PC

MCU 포트로 문자를 보내고 FPGA 포트에서 수신 결과를 확인하므로 서로 다른 두 COM 포트를 열어 사용한다.

핵심 설계

과제5의 문자 입력 처리에 UART3 송신을 추가한다. 문자를 uint8_t형 데이터로 변환한 뒤 다음 함수를 호출한다.

HAL_UART_Transmit(&huart3, &data, 1U, 100U);

UART3로 한 바이트를 전달하며 최대 대기 시간은 100 ms이다. 반환값은 HAL_StatusTypeDef 변수에 저장하고, 받은 문자·바이트 값과 함께 MCU VCP에 출력한다.

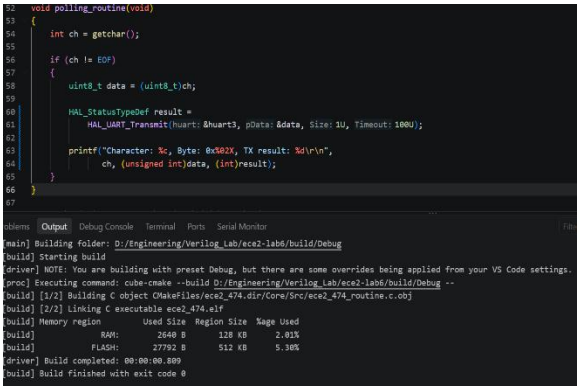


그림 3: STM32 과제 6 코드 및 CMake 빌드 결과

FPGA 테스트벤치 검증 방법

테스트벤치는 0x41, 0x5A, 0x0A를 순서대로 입력하고, FPGA TX에서 복원한 바이트가 원래 입력과 일치하는지 검사한다. LED 버스가 수신 바이트와 같은 값인지도 확인한다.

예상 결과

TB 입력 바이트	의미	예상 Echo	예상 LED 버스
0x41	문자 A	0x41	0x41
0x5A	문자 Z	0x5A	0x5A
0x0A	줄바꿈	0x0A	0x0A

사전 검증 결과

테스트벤치 실행 결과 LAB3_UART_ECHO_PASS checks=3을 확인하였다. 메시지의 LAB3 표기는 재사용한 테스트벤치 이름이며, 이번 과제6의 UART Echo 검증 결과로 사용하였다.

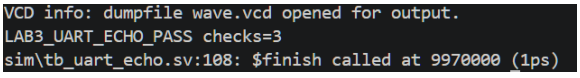


그림 4: STM32 과제 6 UART Verilog PASS 결과

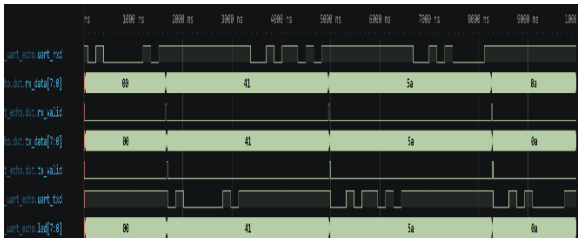


그림 5: STM32 과제 6 UART Verilog 파형 결과

rx_data가 초기값 00에서 41, 5A, 0A로 바뀌고 각 바이트 수신 시점에 rx_valid 펄스가 발생한다. 이후 tx_data에도 같은 값이 전달되며, tx_valid 발생 뒤 uart_txd에 직렬 프레임이 나타난다. LED 버스 역시 41, 5A, 0A로 변하여 수신 데이터가 반영됨을 확인하였다. 따라서 세 입력에 대한 수신·재송신과 LED 표시가 예상 결과와 일치한다.

링크: 과제6 소스 코드

링크: 과제6 FPGA 증빙

6. STM32 과제 7

설계 목표

과제6의 FPGA 수신 처리를 확장하여 받은 문자에 따라 지정 LED를 제어한다. 문자 0은 끄기, 1은 켜기, 2는 상태 반전으로 처리하고 그 밖의 문자는 기존 상태를 유지한다. 받은 명령은 FPGA UART 출력으로도 확인한다.

핵심 설계

MCU는 PC 입력 문자를 UART3로 전달하므로 과제6과 과제7의 MCU 코드는 동일하다. 명령 판별과 LED 상태 변경은 FPGA의 lab6_uart_led에서 수행한다.

rx_valid가 발생하면 rx_data를 case 문으로 비교한다. 0x30이면 led_state에 0, 0x31이면 1을 대입하고, 0x32이면 기존 상태를 반전한다. 다른 값은 상태를 유지한다.

리셋 시 LED 상태는 0이다. 지정 LED는 led[0]이며 나머지 led[7:1]은 0으로 유지한다. 수신 시 UART 송신기가 준비되어 있으면 같은 바이트로 Echo 전송도 요청한다.

```
void polling_routine(void)
{
    int ch = getchar();

    if (ch != EOF)
    {
        uint8_t data = (uint8_t)ch;

        HAL_StatusTypeDef result =
            HAL_UART_Transmit(huart: &huart3, pData: &data, Size: 1U, Timeout: 100U);

        printf("Character: %c, Byte: 0x%02X, TX result: %d\r\n",
               ch, (unsigned int)data, (int)result);
    }
}
```

```
Output  Debug Console  Terminal  Ports  Serial Monitor
[main] Building folder: D:/Engineering/Verilog_Lab/ece2-lab6/build/Debug
[build] Starting build
[driver] NOTE: You are building with preset Debug, but there are some overrides being applied from your VS Code settings.
[proc] Executing command: cube-cmake --build D:/Engineering/Verilog_Lab/ece2-lab6/build/Debug --
[build] [1/2] Building C object CMakeFiles/ece2_474.dir/Core/Src/ece2_474_routine.c.obj
[build] [2/2] Linking C executable ece2_474.elf
[build] Memory region         Used Size  Region Size  %age Used
[build]             RAM:        2640 B       128 KB      2.01%
[build]           FLASH:       27792 B       512 KB      5.30%
[driver] Build completed: 00:00:00.809
[build] Build finished with exit code 0
```

그림 6: STM32 과제 7 코드 및 CMake 빌드 결과

예상 결과

입력 문자	바이트 값	FPGA LED 동작
0	0x30	끄기
1	0x31	켜기
2	0x32	현재 상태 반전

FPGA 테스트벤치 검증 방법

테스트벤치는 다음 순서로 문자를 입력한다.

0 → 1 → 1 → 2 → 2 → A → 0

각 입력에 대해 Echo 바이트와 led[0]의 예상 상태를 비교하고, 나머지 LED가 0으로 유지되는지 확인한다. UART 분주값은 과제6과 동일한 DIV=8이고 TB 클록 주기는 20 ns이다.

입력 순서	문자	예상 led[0]	확인 목적
1	0	0	OFF
2	1	1	ON
3	1	1	ON 유지
4	2	0	반전
5	2	1	반전
6	A	1	유지
7	0	0	OFF 복귀

사전 검증 결과

LAB6_ASSIGNMENT7_UART_LED_PASS checks=7을 확인하였다.

```
VCD info: dumpfile wave.vcd opened for output.
LAB6_ASSIGNMENT7_UART_LED_PASS checks=7
sim\tb_uart_led.sv:118: $finish called at 23311000 (1ps)
```

그림 7: STM32 과제 7 led Verilog PASS 결과

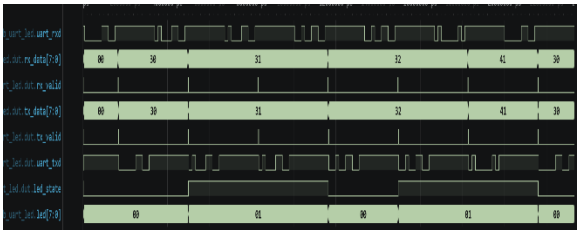


그림 8: STM32 과제 7 led Verilog 파형 결과

처음 30을 받은 후 LED는 00이다. 첫 번째 31에서 01로 바뀌고 두 번째 31에서는 01을 유지한다. 첫 번째 32에서는 00, 두 번째 32에서는 01이 된다. 41, 즉 문자 A에서는 01을 유지하고 마지막 30에서 00으로 돌아간다. 따라서 반복된 1은 ON 유지, 반복된 2는 매번 상태 반전, 기타 문자는 상태 유지라는 교안의 요구와 일치한다.

링크: 과제7 소스 코드

링크: 과제7 STM 빌드 증빙

링크: 과제7 FPGA 증빙

7. FPGA 실습 1 · 서보모터 제어

설계 목표

버튼 입력에 따라 서보 위치 명령의 펄스 폭을 변경하고, 20 ms 프레임에서 1~2 ms의 HIGH 펄스를 생성한다. 초기 펄스 폭은 1.5 ms이고 유효 입력마다 0.1 ms씩 변경한다.

핵심 설계

servo_position은 위치 명령에 해당하는 pulse_us를 저장한다. 리셋 시 1500으로 초기화하고, 좌측 입력만 발생하면 100씩 감소시키며 우측 입력만 발생하면 100씩 증가시킨다. 범위는 1000~2000 μs로 제한하고 양쪽 입력이 동시에 발생하면 유지한다.

```
always @(posedge clk_50mhz)
    if(rst_p) pulse_us<=1500;
    else if(left_pulse && !right_pulse && pulse_us>1000)
        pulse_us<=pulse_us-100;
    else if(right_pulse && !left_pulse && pulse_us<2000)
        pulse_us<=pulse_us+100;
```

그림 9: 서보모터 제어 핵심 코드1

```
always @(posedge clk_50mhz) begin
    if(rst_p) begin
        count<=0;
        width_cycles<=(CLK_HZ/1000)*3/2;
    end
    else if(count==FRAME_CYCLES-1) begin
        count<=0;
        // 프레임 경계에서만 폭을 바꿔 중간 펄스가 잘리지
        width_cycles<=pulse_us*(CLK_HZ/1000)/1000;
    end
    else
        count<=count+1;
end

assign servo=!rst_p && (count<width_cycles);
```

그림 10: 서보모터 제어 핵심 코드2

테스트벤치 검증 방법

테스트벤치는 servo_position과 servo_pwm을 연결하고 CLK_HZ=100000을 적용한다. 실제 TB 클록 주기는 10 ns이므로 한 프레임은 2000클록, 즉 20 μs이다. HIGH 시간은 100·150·200클록에 대응하는 1·1.5·2 μs이다.

이는 보드의 20 ms 프레임과 1·1.5·2 ms 펄스를 실제 시간으로 재현한 것이 아니라, 시뮬레이션 시간을 줄여 계수 관계를 검사한 것이다.

예상 결과

입력 조건	예상 펄스 폭	예상 동작
리셋	1.5 ms	초기 위치 명령
초기 상태에서 왼쪽 1회	1.4 ms	펄스 폭 감소
이후 오른쪽 1회	1.5 ms	초기 폭으로 복귀

검증 결과

저장한 파형에서는 us와 width_cycles를 십진수로 표시하였다. us는 1500→1000→2000으로 변하고, 실제 HIGH 계수인 width_cycles는 150→100→200으로 변한다.

us가 바뀐 직후에도 기존 펄스 폭을 유지하는 구간이 있으며, 다음 프레임 경계에서 새 계수가 적용된다. 이후 servo의 HIGH 구간이 새 계수에 따라 줄거나 늘어나는 것을 확인하였다.

```
VCD info: dumpfile wave.vcd opened for output.
PASS servo: 20ms frame, 1/1.5/2ms pulse, limits, reset
sim\tb.sv:97: $finish called at 100036000 (1ps)
```

그림 11: 서보모터 Verilog PASS 결과

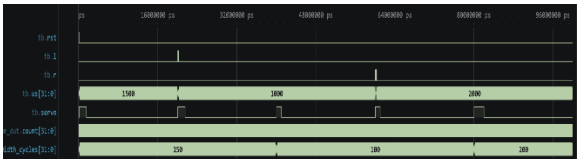


그림 12: 서보모터 제어 시뮬레이션 파형

링크: 서보 모터 소스코드

링크: 서보 모터 증빙자료

8. FPGA 실습 2 · SRAM 읽기·쓰기

설계 목표

주소 0~15에 데이터를 저장하고 같은 주소에서 다시 읽어 일치하는지 확인한다. 외부 데이터 버스는 16비트이며 이번 실습은 주소 하위 4비트와 데이터 하위 4비트를 사용한다.

핵심 설계

sram_bus_control은 읽기·쓰기 요청에 따라 버스 방향과 제어 신호를 결정한다. 쓰기만 요청되면 drive_data=1, 읽기만 요청되면 read_data=1이다. 두 요청이 모두 0이거나 모두 1이면 읽기와 쓰기를 수행하지 않는다. ram_cs, ram_oe, ram_we는 LOW 활성 신호이다

```
assign ram_address={14'b0,address};
assign ram_data=drive_data ? {12'b0,data_in} : 16'bz;
assign led_out=read_data ? {4'b0,ram_data[3:0]} : 8'b0;
```

그림 13: SRAM 핵심 코드1

```
assign drive_data=ctrl_w && !ctrl_r;
assign read_data=ctrl_r && !ctrl_w;

assign ram_cs=!(drive_data || read_data);
assign ram_oe=!read_data;
assign ram_we=!drive_data;
```

그 림 1 4 : S R A M 핵 심 코 드 2

테스트벤치 검증 방법

단순 비동기 SRAM 모델을 연결하여 주소 i에 15-i를 저장한다. 이어서 같은 16개 주소를 순서대로 읽고 버스 값과 LED가 저장값과 일치하는지 검사한다.

대기 상태에서 버스가 Z이고 LED가 0인지 확인하고, 읽기·쓰기 동시 요청에서는 제어 신호가 모두 HIGH이며 버스가 Z인지 검사한다.

예상 결과

주소-16진수	저장, 읽기 데이터-16진수
0,1,2	F, E, D
7,8	8,7
E, F	1,0

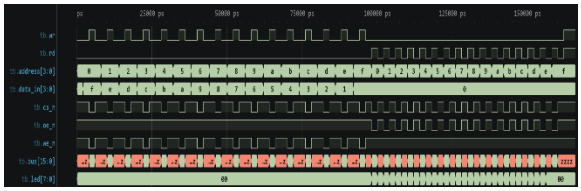
검증 결과

파형의 앞부분에서 address는 0~F로 증가하고 data_in은 F~0으로 감소한다. wr가 HIGH인 구간마다 cs_n과 we_n이 LOW가 되며 oe_n은 HIGH로 유지된다. 쓰기 사이에는 버스가 Z가 되고 LED는 00을 유지한다.

뒷부분에서는 rd 펄스에 맞추어 cs_n과 oe_n이 LOW가 되고 we_n은 HIGH를 유지한다. 읽기 활성 구간에서 메모리 모델의 데이터가 버스와 LED에 나타나고, 읽기를 해제하면 버스는 다시 Z이며 LED는 00이다.

```
VCD info: dumpfile wave.vcd opened for output.
PASS SRAM: 16 addresses, bus release, simultaneous request
sim\tb.sv:97: $finish called at 164000 (1ps)
```

그 림 15: SRAM Verilog PASS 결 과



그 림 1 6 : S R A M 시 뮬 레 이 션 파 형

링크: SRAM 관련 전체 소스

링크: SRAM 관련 증빙 자료

9. FPGA 실습 3 · DAC

설계 목표

8비트 Ramp 값을 병렬 DAC 인터페이스로 출력하고 채널 선택, 쓰기 펄스, 데이터 유지 및 최댓값 이후의 순환을 검증한다. 실제 아날로그 출력 전압과 파형은 장비 연결 후 확인한다.

핵심 설계

ramp8은 UPDATE_CYCLES마다 update를 발생시키고 8비트 value를 1 증가시킨다.

dac_parallel8은 update가 발생한 클록에서 데이터를 data에, 채널을 a_b에 저장하고 we_n을 LOW로 내린다. 다음 클록에서는 we_n을 HIGH로 되돌린다. 같은 클록에서 실행되는 비차단 대입의 특성에 따라 출력부는 해당 갱신 직전의 value를 받아 첫 출력값 0부터 순서대로 전송한다.

최상위 dac는 두 모듈을 연결하고 led_out=dac_d로 DAC 출력 데이터를 LED에도 표시한다. cs_n과 ldac_n은 0으로 고정되어 있다. 실제 DAC 모델의 핀 기능과 전압 관계는 교재에서 확인한다.

```
assign update=(count==UPDATE_CYCLES-1);

always @(posedge clk_50mhz) begin
    if(rst_p) begin
        count<=0;
        value<=0;
    end
    else if(update) begin
        count<=0;
        value<=value+1'b1;
    end
    else
        count<=count+1;
end
```

그 림 1 7 : D A C 핵 심 코 드 1

```
else begin
    we_n<=1;
    if(update) begin
        data<=value;
        a_b<=channel;
        we_n<=0;
    end
end
```

그 림 1 8 : D A C 핵 심 코 드 2

테스트벤치 검증 방법

TB는 10 ns 클록과 UPDATE_CYCLES=4를 사용하여 총 260회의 쓰기를 검사한다. 각 데이터가 writes % 256과 일치하는지, LED가 데이터와 같은지, 쓰기 LOW 구간에서 데이터와 채널이 유지되는지 확인한다.

예상 결과

조건	예상 결과
리셋	디지털 출력값 00, 쓰기 신호 비활성
정상 갱신	DAC 데이터와 LED가 00-01-..-FF로
FF 다음 갱신	00으로 순환
채널 변경	다음 갱신 시 선택 채널 반영

검증 결과

대표 확대 파형은 약 9.1~10.4 μs 구간이며 눈금 표시는 ps 단위이다. dac_d와 led_out의 16진수 값이 E3, E4, ..., FE, FF, 00, 01, 02로 증가한다.

두 버스가 동일하게 변하므로 LED가 DAC 출력 데이터를 표시하며, 8비트 최댓값 FF 다음에 00으로 순환함을 확인할 수 있다.

```
VCD info: dumpfile wave.vcd opened for output.
PASS DAC: 260 writes, wraparound, channel, data hold, reset
sim\tb.sv:78: $finish called at 10436000 (1ps)
```

그림 19: DAC Verilog PASS 결과

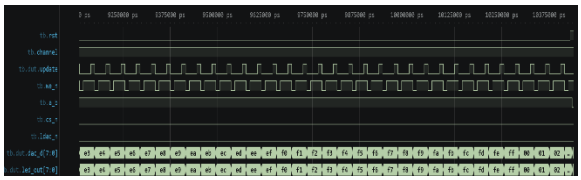


그림 20: DAC 시뮬레이션 파형

링크: DAC 전체 소스

링크: DAC 증빙 자료

10. 사전 검증 결과 및 문제 해결

사전 검증 결과 요약

실습	예상 결과	확인 결과
STM32 과제5	입력 문자와 16진수 바이트 출력	빌드 성공
STM32 과제6	UART3로 수신 문자 전달	빌드 성공
FPGA 과제6	수신 바이트 재송신 및 LED 표시	TB PASS
STM32 과제7	과제 6의 전달 코드 사용	빌드 성공
FPGA 과제7	문자 명령에 따른 LED 제어	TB PASS
서보 실습 1	펄스 폭 증감, 범위 제한, 리셋	TB PASS
SRAM 실습2	저장값 읽기, 버스 해제, 동시 요청 처리	TB PASS
DAC 실습 3	데이터 증가, 순환, 채널 선택, 리셋	TB PASS

검증 시 유의 사항

테스트벤치는 실행 시간을 줄이기 위해 분주값과 계수 조건을 축소하였다. 따라서 저장된 파형의 시간축을 실제 보드의 동작 시간으로 해석하지 않고, 입력과 출력 및 계수 관계를 기준으로 비교한다. 빌드 성공과 테스트벤치 PASS는 각각 프로그램 생성과 디지털 논리 검증 결과이며, 실제 장치 동작은 보드에서 별도로 확인한다.

11. 실험 당일 검증 계획

STM32는 과제별 프로그램을 빌드하고 보드에 다운로드한 뒤 실행을 재개한다. UART 통신에서는 두 보드의 GND를 공통으로 연결하고 송수신 핀과 통신 속도를 확인한다.

FPGA는 실습별 RTL과 XDC를 등록하고 실제 회로를 Design Top으로 지정한다. 보드의 클록을 50 MHz로 설정한 뒤 합성·구현·비트스트림 생성 및 다운로드를 수행한다.

실습	실험 당일 확인할 항목
과제5	1, A, a 입력과 0x31,0x41,0x61
과제6	STM32 입력과 FPGA echo의 일치
과제7	0, 1, 2 및 기타 문자에 따른 LED 상태
서보	버튼별 위치 변화, 범위 제한, 리셋
SRAM	같은 주소에 쓴 값과 읽은 값의 일치
DAC	선택 채널의 전압 변화와 반복 동작

12. 결론

본 실험을 준비하면서 UART를 이용한 문자 입출력과 STM32에서 FPGA로 데이터를 전달하는 구조를 확인하였다. 과제 5~7의 STM32 프로그램은 정상적으로 빌드되었으며, FPGA의 echo 및 LED 명령 처리 회로는 테스트벤치 검사를 통과하였다.

FPGA 개별 실습에서는 서보 제어 펄스의 폭 변경, SRAM의 저장·읽기와 버스 해제, DAC 데이터의 증가와 순환을 확인하였다. 세 실습 모두 테스트벤치 PASS와 대표 파형을 확보하였다.

현재 사전 검증 범위는 STM32의 컴파일·링크와 FPGA의 디지털 시뮬레이션이다. 실험 당일에는 실제 UART 통신과 장치별 보드 동작을 확인하여 예상 결과와 비교한다.

참고 문헌

[1] Walt Kester, "Basic DAC Architectures II: Binary DACs," *MT-015 Tutorial*, Rev. A, Analog Devices, October 2008, pp. 1, 4–5. https://www.analog.com/media/en/training-

seminars/tutorials/MT-015.pdf